React Router: Complete Guide

3.1 – React Router: Complete Guide

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"What is React Router? Why do we need it?"	🟢	★★★★★
"What is the difference between BrowserRouter and HashRouter?"	🟡	★★★★☆
"How do you get URL parameters in React?"	🟢🟡	★★★★★
"How do you navigate programmatically?"	🟡	★★★★☆
"What is the difference between Link and NavLink?"	🟢	★★★☆☆
"How do you handle 404 pages?"	🟢	★★★☆☆

Why React Router Exists

React is a single-page application (SPA) framework. When the user clicks a navigation link, you don't want the browser to reload the entire page from the server. You want to swap out components while keeping the URL in sync. React Router handles this mapping between URLs and components.

Without React Router, you'd have to manually listen to `window.location`, parse the URL yourself, and conditionally render components based on it. React Router does all of this with a clean, declarative API.

Basic Setup (React Router v6)

React Router v6 is the current standard. If you see `Switch`, `component={}`, or `exact` in code, that's v5 syntax. Interviewers may ask about both, but always default to v6 in new code.

JSX Copy

```jsx
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
        <Link to="/contact">Contact</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
```

```
        </Routes>
      </BrowserRouter>
    );
  }
```

The `path="*"` route is a catch-all for 404 pages. It matches any URL that doesn't match the routes above it.

BrowserRouter vs HashRouter

	BrowserRouter	HashRouter
URL format	`/about`	`/#/about`
Requires server config?	Yes (all routes must return index.html)	No
Looks cleaner?	Yes	No (has `#` in URL)
SEO friendly?	Better	Worse

use

Use `BrowserRouter` unless you're deploying to a static host that doesn't support URL rewrites (like GitHub Pages without configuration).

Core Hooks

useParams: Reading URL Parameters

JSX Copy

```jsx
// Route definition
<Route path="/users/:userId" element={<UserProfile />} />

// Component
function UserProfile() {
  const { userId } = useParams();
  return <h1>User ID: {userId}</h1>;
}

// URL: /users/42  ⟶  userId = "42"
```

Parameters are always strings. If you need a number, convert it: `Number(userId)`.

useNavigate: Programmatic Navigation

JSX Copy

```jsx
function LoginForm() {
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    const success = await login(credentials);
```

```
      navigate('/dashboard');                // Go to dashboard
      // navigate('/dashboard', { replace: true });  // Replace current hi
      // navigate(-1);                       // Go back one page
    }
  };

  return <form onSubmit={handleSubmit}>...</form>;
}
```

useSearchParams: Query String Parameters

JSX Copy

```jsx
// URL: /products?category=shoes&sort=price

function ProductList() {
  const [searchParams, setSearchParams] = useSearchParams();

  const category = searchParams.get('category');  // "shoes"
  const sort = searchParams.get('sort');          // "price"

  const updateSort = (newSort) => {
    setSearchParams({ category, sort: newSort });
  };

  return (
    <div>
      <p>Category: {category}, Sort: {sort}</p>
      <button onClick={() => updateSort('name')}>Sort by Name</button>
    </div>
  );
}
```

useLocation: Current Location Object

JSX Copy

```
  const location = useLocation();
  // location.pathname = "/users/42"
  // location.search = "?tab=posts"
  // location.state = { from: 'dashboard' }  (passed via navigate)

  return <p>You are at: {location.pathname}</p>;
}
```

Nested Routes and Outlet

Nested routes let you build layouts where some parts of the page change and other parts stay constant.

JSX Copy

```jsx
function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Layout />}>
          <Route index element={<Home />} />
          <Route path="about" element={<About />} />
          <Route path="users" element={<Users />}>
            <Route path=":userId" element={<UserProfile />} />
          </Route>
        </Route>
      </Routes>
    </BrowserRouter>
  );
}

function Layout() {
  return (
    <div>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
        <Link to="/users">Users</Link>
```

```
        <Outlet />

        <footer>Footer stays on every page</footer>
      </div>
    );
}
```

`<Outlet />` is the placeholder where child route components render. The Layout component wraps all pages, so the nav and footer persist while the main content swaps.

Link vs NavLink

`Link` is for basic navigation. `NavLink` is for navigation links that need to show an "active" state.

JSX Copy

```jsx
// Link: simple navigation
<Link to="/about">About</Link>

// NavLink: adds "active" class automatically when the route matches
<NavLink
  to="/about"
  className={({ isActive }) => isActive ? 'nav-link active' : 'nav-link'}
>
  About
</NavLink>
```

Use NavLink for navigation menus and sidebars where highlighting the current page matters.

---

v5 vs v6 Quick Reference

Feature	v5	v6
Route component	`<Route component={Home} />`	`<Route element={<Home />} />`
Wrapping routes	`<Switch>`	`<Routes>`
Exact match	`<Route exact path="/">`	Exact by default
Redirect	`<Redirect to="/login" />`	`<Navigate to="/login" replace />`
Access params	`props.match.params.id`	`useParams()`
Programmatic nav	`useHistory().push('/path')`	`useNavigate()('/path')`

Common Mistakes

Mistake 1: Putting Routes outside BrowserRouter. Everything that uses React Router hooks or components must be inside a `<BrowserRouter>`.

Mistake 2: Using `<a href>` instead of `<Link to>`. An `<a>` tag causes a full page reload. `<Link>` performs client-side navigation without reloading.

Mistake 3: Forgetting the catch-all route. Without `path="*"`, unmatched URLs show a blank page instead of a 404 component.

Mistake 4: Not handling trailing slashes. `/about` and `/about/` can be different routes. Be consistent.

🔥 Output Question

JSX Copy

```jsx
<Routes>
  <Route path="/users" element={<Users />} />
  <Route path="/users/:id" element={<UserDetail />} />
  <Route path="*" element={<NotFound />} />
</Routes>
```

What renders at URL `/users/42`?

`<UserDetail />` renders. React Router v6 picks the most specific match. `/users/:id` is more specific than `/users` for the URL `/users/42`. No `exact` keyword needed in v6.

🗣️ Interview Answer

> "React Router is a client-side routing library for React SPAs. It maps URLs to components without full page reloads. In v6, you define routes with `<Routes>` and `<Route>`, use `useParams` for URL parameters, `useNavigate` for programmatic navigation, and `useSearchParams` for query strings. Nested routes with `<Outlet>` let you build persistent layouts. BrowserRouter uses clean URLs and needs server-side URL rewriting, while HashRouter uses the hash fragment and works on any static host."

Key Takeaways

- React Router maps URLs to components for SPA navigation.
- **BrowserRouter** for clean URLs (needs server config). **HashRouter** for static hosts.
- `<Link>` for navigation (not `<a>`). `<NavLink>` for active-state styling.
- `useParams` for URL params, `useNavigate` for programmatic nav, `useSearchParams` for query strings.
- `<Outlet />` renders child routes inside a layout component.
- `path="*"` catches unmatched URLs for 404 pages.
- v6 uses `element={}` (not `component={}`), `<Routes>` (not `<Switch>`), and matching is exact by default.

QUICK CHECK — no marks, just for you.

1. A route is defined as <Route path="/users/:userId" element={<UserProfile />} /> and the user visits /users/42. Inside UserProfile, what does const { userId } = useParams() give you?

- ◯ The string ":userId", because useParams returns the pattern, not the value
- ◯ undefined, because useParams only works with query strings
- ◯ The number 42, because React Router coerces numeric segments
- ◯ The string "42", because route parameters are always strings

Check answer

2. You are deploying a React SPA to a static file host that cannot be configured to rewrite all URLs to index.html. Based on the lesson, which router should you reach for and why?

- ◯ BrowserRouter, because HashRouter cannot support URL parameters like /users/:id
- ◯ Either one, since routing is entirely client-side and the server is never involved

HashRouter, because it keeps routing in the hash fragment and needs no server configuration

BrowserRouter, because it works on any host and only HashRouter needs server rewrites

Check answer

3. Given these v6 routes, what renders at the URL /users/42? <Routes> <Route path="/users" element={<Users />} /> <Route path="/users/:id" element={<UserDetail />} /> <Route path="*" element={<NotFound />} /> </Routes>

UserDetail, because v6 picks the most specific match

NotFound, because /users/:id needs an exact prop to match

Users, because /users is listed first and matches as a prefix

Both Users and UserDetail, since every matching route renders

Check answer

‹ Previous
useLayoutEffect vs useEffect

Next ›
Protected Routes & Authentication Flow

LEARN MORE

ARTICLES & DOCS

React Router — Routing
reactrouter.com

React Router — home & docs

reactrouter.com

VIDEOS

React.js Interview Kit



React Router v6 in 45 minutes

YouTube



A complete guide to routing in React

YouTube



React Router 6 for beginners

YouTube

Hand-picked references for this lesson.